SLGA™ Complete Standards & Modules Index

SLGA™ Complete Standards & Modules Index

---------- Forwarded message ---------

From: Knowenger <knowenger@gmail.com> Date: Fri, Oct 16, 2026, 9:14 AM Subject: SLGA™ Complete Standards & Modules Index To: Miroslav Pis <swissstaffingagency@gmail.com>, <Gutflex@gmail.com>, < originopenfoundation@gmail.com>

SLGA™ Complete Standards & Modules Index Software Lifecycle Governance

Architecture — Software Lifecycle Governance Layer Official Software Lifecycle Governance Architecture Map for Software Products, Applications, Components, Services, Platforms, AI Systems, Autonomous Software Agents, Digital Infrastructure, Embedded Software, and Future Software Systems

This document serves as the official OOF® Software Lifecycle Governance

architecture map and structural index used for Software Lifecycle Governance design, Software Lifecycle navigation, software governance diagnostics, architecture development, Software Lifecycle Governance assessment, lifecycle state analysis, software change and retirement governance, continuous lifecycle control, and interoperability across software products, applications, components, services, platforms, AI systems, autonomous software agents, digital infrastructure, embedded software, and future software systems.

SLGA™ consists of 11 Parent Standards and 55 Core Modules.

Additional standards and modules may be added when new governable Software Lifecycle spaces are identified. However, the standards and modules listed below form the core structural backbone of SLGA™.

• Software Requirements Standard (SRS)

Governed Space: Software Requirements Governance

Core Question: How can software purpose, requirements, scope, changes, and traceability remain identifiable and governable throughout the software lifecycle?

Modules:

- SPDM — Software Purpose Definition Module

- RSPM — Requirements Specification Module - SSMM — Software Scope Management Module - RCCM — Requirements Change Control Module - RTM — Requirements Traceability Module

Software Creation Standard (SCS)

Governed Space: Software Creation Governance

Core Question: How can software creation remain identifiable, methodologically controlled, attributable, and traceable across human, automated, and autonomous creation processes?

Modules:

- DMSM — Development Method Selection Module

- CRIM — Creator Identification Module - ASCCM — Automated Software Creation Control Module - SACM — Software Artifact Creation Module - DTM — Development Traceability Module

Software Dependency Standard (SDS)

Governed Space: Software Dependency Governance

Core Question: How can software dependencies remain identifiable, origin-traceable, suitable, change-controlled, and governable throughout the software lifecycle?

Modules:

- CIM — Component Inventory Module

- DOTM — Dependency Origin Tracking Module - DSM — Dependency Selection Module - CICM — Component Integration Control Module - DCRM — Dependency Change & Risk Management Module

Software Configuration Standard (SCFS)

Governed Space: Software Configuration Governance

Core Question: How can software configurations, builds, artifacts, and versions remain identifiable, controlled, reproducible where required, and traceable across lifecycle changes?

Modules:

- CBM — Configuration Baseline Module

- BMM — Build Management Module - AVIM — Artifact & Version Identification Module - CCCM — Configuration Change Control Module - BREM — Build Reproducibility & Equivalence Module

Software Quality Standard (SQS)

Governed Space: Software Quality Governance

Core Question: How can software quality requirements, evaluation criteria, testing coverage, defects, and resulting quality conditions remain governed through identifiable evidence?

Modules:

- QRM — Quality Requirements Module

- QCDM — Quality Criteria Definition Module - TCMM — Test Coverage Management Module - DFM — Defect Management Module - QCAM — Quality Condition Assessment Module

Software Security Standard (SSS)

Governed Space: Software Security Governance

Core Question: How can software security requirements, exposures, testing, vulnerabilities, and remediation remain identifiable, controlled, and verifiable throughout the software lifecycle?

Modules:

- SRQM — Security Requirements Module

- SEAM — Security Exposure Assessment Module - STCM — Security Testing Control Module - VMM — Vulnerability Management Module - SRCM — Security Remediation Control Module

Software Release Standard (SRLS)

Governed Space: Software Release Governance

Core Question: Under what governed conditions can an identifiable software release candidate become ready, legitimately authorized, recorded, and subject to withdrawal where required?

Modules:

- RCMM — Release Candidate Management Module

- RRM — Release Readiness Module - RAM — Release Authorization Module - RRMM — Release Record Management Module - RWM — Release Withdrawal Module

Software Deployment Standard (SDPS)

Governed Space: Software Deployment Governance

Core Question: How can software deployment remain planned, authorized, operationally identifiable, monitored, and recoverable across actual deployment environments?

Modules:

- DPM — Deployment Planning Module

- DAM — Deployment Authorization Module - PVCM — Production Version Control Module - DSMM — Deployment State Monitoring Module - RRVM — Rollback & Recovery Module

Software Change Management Standard (SCMS)

Governed Space: Software Change Management Governance

Core Question: How can software changes remain requested, impact-assessed, coordinated, routed through applicable lifecycle governance, and legitimately closed?

Modules:

- CRMM — Change Request Management Module

- CIAM — Change Impact Assessment Module - CHCM — Change Coordination Control Module - LRECM — Lifecycle Re-entry Control Module - CCLM — Change Closure Management Module

Software Retirement Standard (SRTS)

Governed Space: Software Retirement Governance

Core Question: How can software be legitimately, safely, and verifiably retired when its operational use, support obligations, dependencies, data, users, authority relationships, and residual responsibilities may continue beyond the decision to discontinue it?

Modules:

- RPM — Retirement Planning Module

- ESMM — End-of-Support Management Module - SMM — Software Migration Module - RDMM — Residual Dependency Management Module - RCCLM — Retirement Closure Control Module

Software Lifecycle Control Standard (SLCS)

Governed Space: Software Lifecycle Control Governance

Core Question: How can software remain identifiable, controllable, and governable throughout its lifecycle when its states, transitions, actors, dependencies, operational conditions, authority relationships, and governance requirements continuously change?

Modules:

- LSCCM — Lifecycle State Consistency Control Module

- LTCM — Lifecycle Transition Control Module - LEMM — Lifecycle Exception Management Module - LCTCM — Lifecycle Control Transfer & Continuity Module - LCAM — Lifecycle Control Assessment Module

SLGA™ Architecture Summary

Parent Standards: 11

Core Modules: 55

Core Governed Software Lifecycle Spaces

- Software Requirements

- Software Creation - Software Dependencies - Software Configuration - Software Quality - Software Security - Software Release - Software Deployment - Software Change Management - Software Retirement - Software Lifecycle Control

SLGA™ Navigation Layer Core Questions Map

SRS → How can software purpose, requirements, scope, changes, and traceability remain identifiable and governable throughout the software lifecycle?

SCS → How can software creation remain identifiable, methodologically controlled, attributable, and traceable across human, automated, and autonomous creation processes?

SDS → How can software dependencies remain identifiable, origin-traceable, suitable, change-controlled, and governable throughout the software lifecycle?

SCFS → How can software configurations, builds, artifacts, and versions remain identifiable, controlled, reproducible where required, and traceable across lifecycle changes?

SQS → How can software quality requirements, evaluation criteria, testing coverage, defects, and resulting quality conditions remain governed through identifiable evidence?

SSS → How can software security requirements, exposures, testing, vulnerabilities, and remediation remain identifiable, controlled, and verifiable throughout the software lifecycle?

SRLS → Under what governed conditions can an identifiable software release candidate become ready, legitimately authorized, recorded, and subject to withdrawal where required?

SDPS → How can software deployment remain planned, authorized, operationally identifiable, monitored, and recoverable across actual deployment environments?

SCMS → How can software changes remain requested, impact-assessed, coordinated, routed through applicable lifecycle governance, and legitimately closed?

SRTS → How can software be legitimately, safely, and verifiably retired when its operational use, support obligations, dependencies, data, users, authority relationships, and residual responsibilities may continue beyond the decision to discontinue it?

SLCS → How can software remain identifiable, controllable, and governable throughout its lifecycle when its states, transitions, actors, dependencies, operational conditions, authority relationships, and governance requirements continuously change? Architectural Position

SLGA™ governs the complete Software Lifecycle Governance lifecycle from

Software Requirements and Creation through Dependencies, Configuration, Quality, Security, Release, Deployment, Change Management, and Retirement, with Software Lifecycle Control operating continuously across all lifecycle stages.

The architecture progresses:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance: Software Lifecycle Control

Together these standards govern why software is required, how its requirements and scope are established, how software is created, which dependencies it incorporates, how configurations and versions remain controlled, how quality and security conditions are evaluated, when release becomes legitimate, how deployment is authorized and verified, how changes return to applicable lifecycle governance, how software is retired, and how lifecycle states, transitions, exceptions, control transfers, and control effectiveness remain governed across the complete software lifecycle.

Its complete lifecycle progresses:

Software Purpose → Requirements → Creation → Dependency Identification &

Control → Configuration & Build Control → Quality Assessment → Security Assessment → Release Readiness & Authorization → Deployment Authorization & Verification → Operational Change Governance → Retirement & Closure

Software Release therefore does not complete the software lifecycle.

Software Release ≠ Software Lifecycle Completion.

Software Deployment also does not complete the software lifecycle.

Software Deployment ≠ Software Lifecycle Completion.

Where material software requirements, artifacts, dependencies, configurations, quality conditions, security conditions, or operational circumstances change:

Governed Software → Change Request → Change Impact Assessment → Applicable

Lifecycle Re-Entry → Renewed Governance → Release / Deployment where applicable → Continued Lifecycle Governance

Where software retirement becomes applicable:

Retirement Planning → End-of-Support Management → Software Migration where applicable → Residual Dependency Management → Retirement Closure Control

Where a material lifecycle control deficiency is identified:

Lifecycle Control Assessment → Control Deficiency → Corrective Governance

→ Applicable Lifecycle Reassessment → Control Verification → Updated Lifecycle Control State

This recursive structure allows Software Lifecycle Governance to remain

responsive to software changes, dependency changes, configuration changes, security remediation, quality findings, release withdrawal, deployment rollback, operational incidents, control transfers, exception changes, retirement conditions, and new materially relevant evidence.

Re-enter the earliest materially affected governance space.

Reassess what materially changed. Preserve what remains valid.

Lifecycle re-entry must preserve software identity and historical traceability. Cross-Architecture Boundary

SLGA™ ↔ GOA™

GOA™ governs the constitutional structure, authority, boundaries,

delegation, validation, and continuity of governance. SLGA™ governs the software lifecycle under applicable governance requirements and legitimate authority.

General Governance Authority ≠ Software Lifecycle Authorization.

SLGA™ ↔ ORA™

ORA™ governs what is actually occurring in Operational Reality. SLGA™ governs how software lifecycle requirements, states, transitions, and controls apply to that operational reality.

Declared Software State ≠ Actual Operational State.

SLGA™ ↔ AGA™

AGA™ governs responsibility, accountability, attribution, authority, and

related accountability relationships. SLGA™ governs software lifecycle requirements and controls involving those relationships.

Responsibility Assigned ≠ Lifecycle Control Effective.

SLGA™ ↔ ADIT®

ADIT® governs evidential observation, preservation, verification,

correlation, examination, and auditability. SLGA™ governs software lifecycle conditions, decisions, and controls to which relevant evidence applies.

Evidence Recorded ≠ Lifecycle Requirement Satisfied.

SLGA™ ↔ VALIDOS™

VALIDOS™ governs whether a validation basis sufficiently supports a

Validation Determination. SLGA™ governs software lifecycle decisions and requirements that may rely upon validated information.

Validation Determination ≠ Release Authorization.

SLGA™ ↔ INTEGROS®

INTEGROS® governs integrity across objects, states, evidence, and

processes. SLGA™ governs software lifecycle requirements and controls.

General Integrity Governance ≠ Software Lifecycle Governance.

SLGA™ ↔ OBIDENITY®

OBIDENITY® governs Origin, Identity, Ownership, Authority, Permission,

Responsibility, Provenance, Transfer, Continuity, and Auditability. SLGA™ governs software objects, artifacts, versions, configurations, and their lifecycle relationships.

Software Identity ≠ Software Lifecycle State.

Software Origin ≠ Software Release Origin by default.

SLGA™ ↔ AIG®

AIG® governs AI behavioral governance and behavioral integrity. SLGA™

governs the software lifecycle of AI-enabled software systems.

AI Behavior ≠ Software Lifecycle Governance.

AI Behavioral Failure ≠ Software Lifecycle Control Failure by default.

SLGA™ ↔ ASGA™

ASGA™ governs autonomous-system authority, execution, constraints,

coordination, intervention, escalation, and operational governability. SLGA™ governs software lifecycle requirements, decisions, and controls applicable to autonomous software systems.

Autonomous Execution Capability ≠ Software Lifecycle Authority.

Autonomous Action ≠ Authorized Lifecycle Transition.

SLGA™ ↔ SIMULOS®

SIMULOS® governs simulation purpose, structure, execution, outputs, and

lifecycle. SLGA™ governs actual software lifecycle objects, conditions, decisions, and states.

Simulated Deployment ≠ Actual Deployment.

Simulated Control Success ≠ Verified Operational Control Effectiveness.

SLGA™ ↔ Software Engineering Systems

SLGA™ governs the software lifecycle. Software engineering systems

govern development activities, technical implementation, coding, testing execution, build operations, deployment execution, maintenance operations, and related engineering workflows.

Software Engineering Execution ≠ Software Lifecycle Governance.

Software Created ≠ Software Governed.

SLGA™ ↔ Software Security Systems

SLGA™ governs the software lifecycle, including software security

requirements, exposure assessment, testing control, vulnerability management, and remediation control. Specialized software security systems perform technical detection, analysis, protection, testing, and remediation activities.

Security Tool Execution ≠ Security Governance Completion.

Vulnerability Remediation Applied ≠ Remediation Effectiveness Verified. Software Identity & Configuration Structure

SLGA™ does not assume that one software product corresponds to one version, one configuration, or one operational state.

A single software product may contain:

multiple components

multiple dependencies

multiple builds

multiple versions

multiple configurations

multiple release candidates

multiple authorized releases

multiple deployment instances

multiple operational environments

multiple simultaneous lifecycle states

or:

multiple independently changing software services

Each software object may possess distinct lifecycle attributes while remaining connected to the broader software product.

Therefore:

Software Product ≠ Software Component.

Software Component ≠ Software Dependency by default.

Software Version ≠ Software Build.

Software Build ≠ Release Candidate.

Release Candidate ≠ Authorized Release.

Authorized Release ≠ Deployed Version.

Deployed Version ≠ Verified Operational Configuration.

One Software Product ≠ One Lifecycle State. Software Change & Lifecycle Re-Entry Structure

Software changes may affect one or more governance spaces.

A change may involve:

Requirements

Creation Methods

Software Artifacts

Dependencies

Configuration

Build

Quality

Security

Release

Deployment

Operational Conditions

Retirement Requirements

or:

Lifecycle Control

SLGA™ does not require every change to restart the complete software lifecycle.

The applicable change path is:

Change Request → Impact Assessment → Change Coordination → Earliest

Materially Affected Governance Space → Applicable Reassessment → Change Closure

A material change may require renewed quality or security evaluation, new release authorization, updated deployment control, or reassessment of lifecycle control effectiveness.

Software Change ≠ Complete Lifecycle Restart by default.

Change Implemented ≠ Change Verified.

Change Verified ≠ Change Closed by default.

Lifecycle Re-Entry ≠ New Software Origin by default. Lifecycle Control Structure

Software Lifecycle Control Standard (SLCS) operates across the complete

SLGA™ architecture.

Its five modules establish:

Lifecycle State Consistency Control

→ whether declared, recorded, observed, and verified software lifecycle

states remain consistent within their applicable scope, context, and time.

Lifecycle Transition Control

→ whether software lifecycle transitions occur under legitimate authority,

applicable preconditions, controlled execution, and verifiable resulting states.

Lifecycle Exception Management

→ whether exceptions to software lifecycle requirements remain legitimate,

bounded, evidence-supported, and accountable.

Lifecycle Control Transfer & Continuity

→ whether software lifecycle control remains effective when authority, responsibility, operational capability, or control arrangements transfer between actors or systems.

Lifecycle Control Assessment

→ whether lifecycle controls exist, are suitably designed, correctly implemented, operationally available, and demonstrably effective.

These mechanisms are cross-cutting.

They do not constitute a mandatory sequential operational phase after

Software Retirement.

Recorded Lifecycle State ≠ Verified Lifecycle State.

Transition Executed ≠ Transition Authorized.

Exception Approved ≠ Requirement Eliminated.

Control Transferred ≠ Control Continuity Verified.

Control Implemented ≠ Control Effective. Retirement & Continuing Governance Structure

Software retirement may involve:

Retirement Planning

End-of-Support Management

Software Migration

Residual Dependency Management

Retirement Closure Control

A retirement decision does not automatically establish retirement completion.

Software may remain operational after support ends.

Dependencies may remain active after a service is withdrawn.

Data, credentials, integrations, users, or contractual obligations may require continuing governance.

Therefore:

Retirement Planned ≠ Retirement Executed.

End of Support ≠ End of Operational Use.

Software Migration ≠ Residual Dependency Elimination.

Operational Withdrawal ≠ Retirement Closure.

Retirement Closure ≠ Historical Evidence Erasure. Foundational Architecture Distinctions

Software Lifecycle Governance ≠ Software Engineering Execution

Software Created ≠ Software Governed

Software Purpose ≠ Verified Requirements

Requirements Documented ≠ Requirements Traceable

Software Generated ≠ Creation Authority Established

Automated Creation ≠ Autonomous Authority

Creator Identified ≠ Creation Authorized

Component Identified ≠ Dependency Suitable

Dependency Available ≠ Dependency Continuity Guaranteed

Dependency Origin Known ≠ Dependency Risk Resolved

Software Product ≠ Software Version

Software Version ≠ Software Build

Software Build ≠ Deployment Instance

Configuration Recorded ≠ Configuration Verified

Build Reproducibility ≠ Software Quality

Testing Performed ≠ Quality Established

Quality Assessment ≠ Security Assessment

Vulnerability Identified ≠ Vulnerability Remediated

Security Remediation Applied ≠ Security Effectiveness Verified

Release Candidate ≠ Authorized Release

Release Readiness ≠ Release Authorization

Release Authorized ≠ Deployment Authorized

Release Authorized ≠ Actual Deployment Verified

Deployment Executed ≠ Deployment Verified

Deployment Successful ≠ Lifecycle Control Effective

Change Requested ≠ Change Authorized

Change Implemented ≠ Change Closed

Software Change ≠ Complete Lifecycle Restart by default

Lifecycle Re-Entry ≠ New Software Identity by default

Retirement Decision ≠ Retirement Completion

End of Support ≠ End of Operational Use

Software Migration ≠ Residual Dependency Elimination

Operational Retirement ≠ Elimination of Continuing Governance Obligations

Declared Lifecycle State ≠ Verified Lifecycle State

Technical Transition Capability ≠ Legitimate Transition Authority

Transition Executed ≠ Transition Verified

Exception Requested ≠ Exception Approved

Exception Authorized ≠ Requirement Eliminated

Exception Approved ≠ Residual Risk Resolved

Responsibility Assigned ≠ Authority Established

Authority Established ≠ Capability Available

Control Transferred Administratively ≠ Control Continuity Verified

Control Documented ≠ Control Implemented

Control Implemented ≠ Control Effective

Control Activity Recorded ≠ Control Objective Achieved

Agent Instruction ≠ Enforced Authority Boundary

Task Authority ≠ Constituent Action Authority

Current Software State ≠ Complete Software Lifecycle History

Canonical Principle

SLGA™ does not prescribe software development methodologies, replace software engineering, perform software creation, execute testing, release, deployment, change, or retirement operations, independently grant lifecycle authority, replace specialized security or quality requirements, govern general accountability, perform general evidence governance, or substitute for contractual, regulatory, safety, technical, or other domain-specific decision systems. SLGA™ governs the methodological conditions under which identifiable software becomes requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governed through consistent lifecycle states, authorized transitions, bounded exceptions, preserved control continuity, and evidence-supported control effectiveness.

Software may be created by humans or autonomous systems, may operate through multiple versions and configurations, may depend on external components, may undergo repeated lifecycle re-entry, may transfer between responsible actors, may remain operational under exceptional conditions, and may retain dependencies or obligations after retirement. SLGA™ provides the governance architecture for that entire lifecycle while preserving the distinction between declared governance and verified operational control.

SLGA™ governs Software from Requirements to Retirement — with continuous

control across the entire lifecycle.